

Project – IV

Healthcare Analytics

CSCI 2951-O: Foundations of Prescriptive Analytics

Due Date: April 17, 10PM

1 Problem Statement

In challenging times, improving the quality of care, cutting down costs, and enhancing clinical decision support are among the top priorities of Healthcare Analytics. To that end, a long-time goal in health IT has been to use symptoms to drive computation towards a diagnosis. However, finding clearly defined data points that single out a specific syndrome from other similar syndromes with enough specificity to provide a diagnosis has been a paramount challenge.

Thanks to efforts of the International Health Consortium (IHC), large amount of data with regard to positive/negative results of different tests on several diseases are collected and are ready to be harnessed. Given that each test incurs a certain cost to perform, IHC is now posed with the following question: how to select a subset of tests with minimum total cost, such that, there is a test in the subset that can differentiate any two diseases. Once again, CSCI 2951-O experts are called into action!

2 Input

The data collected by IHC can be described by a cost vector $\mathbf{c}$ of size n , and an n by m matrix $\mathbf{A}$, where n is the number of tests, m the number of diseases, and $A_{ij} = 1$ if test i is positive for disease j .

Below is a toy example for illustrative purposes. It is easy to see that checking for “fever” is not useful since it is positive for all diseases. A naïve idea is to test for all the 3 remaining symptoms, with cost equals to 3 assuming tests are unicast. The optimal solution is to check for “rash” and “cough” only, with cost equals to 2.

	Flu	Measles	Chickenpox
Rash	0	1	1
Fever	1	1	1
Cough	1	1	0
Headache	1	0	1

The format of the instance files are as follows:

```
#Tests      (i.e., n)
#Diseases   (i.e., m)
Cost_1 Cost_2 . . . Cost_n
A(1,1) A(1,2) . . . A(1, m)
A(2,1) A(2,2) . . . A(2, m)
. . . . .
A(n,1) A(n,2) . . . A(n, m)
```

For instance, the above toy example can be stored in a file of the form:

```
4
3
1 1 1 1
0 1 1
1 1 1
1 1 0
1 0 1
```

3 Your Task

Your first task is to formulate this optimization problem as an Integer Programming (IP) problem. As discussed in the lectures, an effective way of solving IPs is the Branch and Bound (BnB) technique. Your second task is to implement an optimization solver based on BnB to tackle the given instances. You should use Google OR-Tools as a Linear Programming Solver for the linear relaxations arising in the computation of Lower Bounds, but you *cannot* use it as an Integer Programming Solver.

You are free to choose your favorite programming language to complete the project (as long as you can access OR-Tools for solving LPs). The general outline of BnB leaves room for crafting efficient data structures as well as designing effective search heuristics. You are encouraged to experiment with different strategies and share your findings in the project report.

You can assume that a *reasonable* time limit will be used to test the instances. Reasonable time is defined as the time it takes to leave your desk and get a fresh cup of coffee, which is roughly 5 minutes of running time for each instance on the department machines.

Hint: Although you cannot use OR-Tools as an IP solver, you can use it to verify the value of the optimum solution found by your BnB implementation. For example the optimal value for the instance 25_25.0.25.1.ip is 57.

4 Support Code

We provide a data parser that reads the instance and returns a simple IP instance object. This way, you can immediately start attacking the heart of the problem. The simple IP instance class stores the number of tests (`int numTests`), the number of diseases (`int numDiseases`), the cost of each test (`double[] costOfTest`), and the coefficient matrix **A** (`int[] [] A`).

We provide Python support code. You are free to use the support code as is, tweak it as you see fit, or discard it completely. OR-Tools has C++, C#, Java, Rust, and Go bindings as well. However, we cannot provide support on how to run the solver in those languages.

Assuming you are in the `project` directory,

To compile the support code, type:

```
> ./compile.sh
```

To run the support code in a given instance, type:

```
> ./run.sh input/simple.ip
```

In our running example, the expected output of the support code is:

```
> ./run.sh input/simple.cnf
Number of tests: 4
Number of diseases: 3
Cost of tests: [1.0, 1.0, 1.0, 1.0]
A:
[0, 1, 1]
[1, 1, 1]
[1, 1, 0]
[1, 0, 1]
{"Instance": "simple.ip", "Time": 0.02}
```

5 Output

Given an input file as its first command line argument via the `run` script, your solution should print out the result on its **last line**. You can print other statements before the result but they will be discarded. We suggest you to keep such debug information to a minimum since writing I/O is expensive. We use `stdout` for output and ignore other streams. Your submission will be tested on department's linux machines and it should work without any other software package installation.

The expected output of the last line is in JSON format with “Instance” mapped to the instance name, “Time” to the number of seconds it takes your program to solve this instance (in 2-digit precision), “Result” to the objective value found, and “Solution” to “OPT” to denote optimality.

```
./run.sh input/simple.ip
..running..bla..
..bla..
{"Instance": "simple.ip", "Time": 1.23, "Result": 2, "Solution": "OPT"}
```

Finally, to run a benchmark on all instances in a given input folder using a fixed time limit while collecting the result (i.e. the last line) to a log file, type:

```
> ./runAll.sh input/ 300 results.log
```

Please make sure to follow this convention *exactly*! Failing to do so will cause evaluating your submission incorrectly.

6 Submission

```
/solution
|_ src/
|_ compile.sh
|_ run.sh
|_ runAll.sh
|_ results.log
|_ report.pdf
|_ team.txt
```

- Submit all your source code under the (`/src`) folder. Remember that commenting your code is a good practice so that one can follow the flow of your program at a high-level.
- Submit your (possible updated) `compile` and `run` scripts which compiles your solution and runs it on a given instance respectively. Make sure that the *last line* of your solution output follows the specification above.
- Submit the last line found for all the instance in the input directory in the `results.log`.
- Submit a short report in pdf format (1 or 2 pages max) that contains a brief discussion of your constraint model, decision variables and the constraints used as well as your strategy and observations when implement the branch-and-bound algorithm. Don't forget to write name(s) of the team members and the screenname(s) corresponding to this project. Please also include a note on how much time you spend on this project. This information is completely irrelevant to evaluation and kept solely for statistical purposes.
- Submit the cs login(s) of the team in `team.txt` listing one member per line in lowercase.

Have questions? and/or need clarifications? Please do not hesitate to contact us, your instructor and TAs are here to help. Good luck on your quest for helping IHC!